

Documentação Técnica

Agente de Análise Exploratória de Dados

Arquitetura, artefatos e instruções de implementação

Trabalho Final

Tecnologia em Ciência de Dados – 5º Semestre
Fatec Ourinhos
Prof. Dr. Alex Marino

Sumário

1	Introdução	3
1.1	Visão de alto nível	3
1.2	Stack tecnológica	3
2	Diagrama de arquitetura	4
3	Diagrama de sequência: o que acontece em uma pergunta	5
4	Catálogo de artefatos	6
4.1	Estrutura completa de pastas	6
4.2	Visão por responsabilidade	6
5	Camada 1 – Interface (cli.py)	8
5.1	O que faz	8
5.2	Comandos especiais	8
5.3	Fluxo de execução	8
5.4	Pontos de extensão para os alunos	8
6	Camada 2 – Orquestração	8
6.1	agent/llm_client.py	8
6.1.1	Responsabilidade	8
6.1.2	Por que existe um cliente separado	8
6.1.3	Estrutura	9
6.1.4	Tratamento de erros	9
6.2	agent/agent.py	9
6.2.1	Responsabilidade	9
6.2.2	Estruturas principais	9
6.2.3	Anatomia do loop principal	9
6.2.4	Por que o LLM nunca chama uma tool diretamente	10
6.2.5	O SYSTEM_PROMPT	10
7	Camada 3 – Ferramentas (tools/)	11
7.1	O contrato de uma tool	11
7.2	Por que o retorno é sempre um dict	11
7.3	Catálogo das 8 tools obrigatórias	12
7.4	Como adicionar uma tool extra (passo a passo)	12
7.5	DataState: por que um estado global	13
8	Camada 4 – Avaliação (evaluation/)	13
8.1	O conceito de benchmark	13
8.2	Estrutura do benchmark.json	13
8.3	Distribuição obrigatória das 30 perguntas	13
8.4	Pipeline de execução do benchmark	14
8.5	Métricas calculadas	14
8.6	Limitações dos comparadores automáticos	14
9	Fluxos de uso comuns	14
9.1	Fluxo 1: Sessão interativa	14
9.2	Fluxo 2: Rodar testes unitários	15
9.3	Fluxo 3: Executar o benchmark completo	15
9.4	Fluxo 4: Workflow de desenvolvimento (ciclo curto)	15

10 Modelo de custo e otimização	16
10.1 De onde vem o custo	16
10.2 Onde os tokens vão	16
10.3 Estratégias de otimização	16
11 Glossário	17

1 Introdução

Este documento descreve a **arquitetura técnica** do projeto Agente de EDA e serve de referência para todos os integrantes do grupo. O objetivo é que qualquer aluno, ao precisar trabalhar em qualquer parte do sistema, consiga entender:

- Onde aquela parte se encaixa no todo;
- Quais são suas responsabilidades e seus limites;
- Como ela se comunica com as outras partes;
- Como modificá-la sem quebrar nada.

1.1 Visão de alto nível

O sistema é um **agente conversacional** que recebe perguntas em português sobre um arquivo CSV e responde executando, de forma autônoma, operações de análise exploratória através de chamadas a ferramentas (*tools*) pré-definidas.

A diferença fundamental entre este sistema e um *chatbot* comum é que o LLM não responde diretamente a partir do que “acha” que sabe sobre os dados — ele **precisa chamar tools** para obter os fatos. Isso garante que as respostas sejam baseadas em cálculos reais sobre o dataset, e não em alucinações.

Princípio de design: o LLM faz o *raciocínio* sobre quais operações fazer; as *tools* fazem o cálculo determinístico em pandas. Essa separação torna o sistema testável, auditável e robusto.

1.2 Stack tecnológica

Componente	Tecnologia
Linguagem	Python 3.10+
Manipulação de dados	pandas, numpy
Visualização	matplotlib (backend Agg)
LLM (recomendado)	Anthropic Claude Haiku ou Sonnet
Comunicação com LLM	<i>Function calling</i> (tool use) nativo da API
Interface	CLI baseada em rich (terminal colorido)
Persistência	Arquivos JSON em <code>logs/</code> e PNG em <code>outputs/</code>
Testes	pytest
Configuração	python-dotenv para variáveis de ambiente

2 Diagrama de arquitetura

A figura 1 apresenta a arquitetura completa do sistema, organizada em **quatro camadas**.

1. Camada de interface

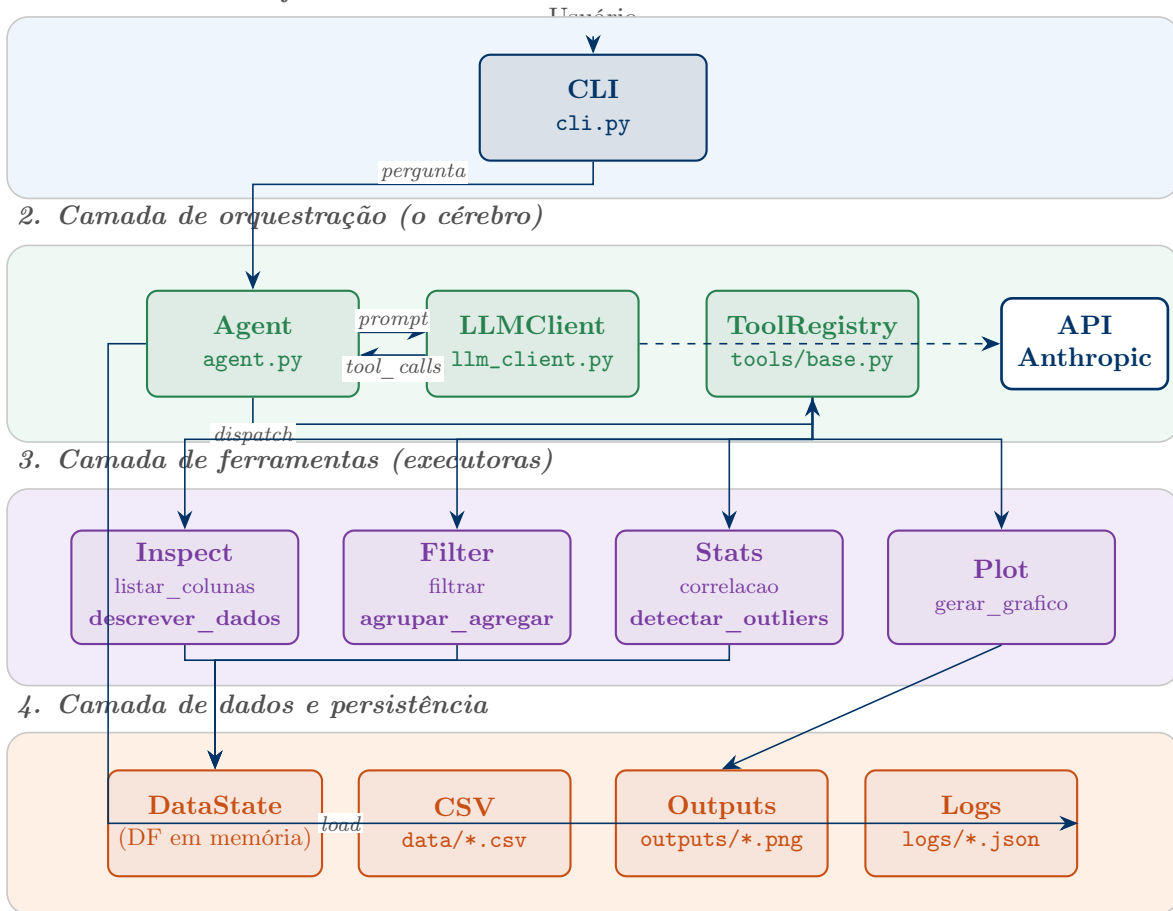


Figura 1: Arquitetura em quatro camadas do agente. Setas sólidas representam fluxo de dados/control dentro do sistema; a seta tracejada vai para a API externa.

Nota técnica

A camada 2 é o “cérebro” do sistema. Tudo o que envolve raciocínio acontece ali. As camadas 3 e 4 são deterministas: dada uma chamada, retornam sempre o mesmo resultado. Essa separação é o que permite testar tudo abaixo da camada 2 sem precisar gastar tokens com o LLM.

3 Diagrama de sequência: o que acontece em uma pergunta

A figura 2 mostra o que ocorre, na ordem cronológica, quando um usuário faz uma pergunta multi-passo ao agente.

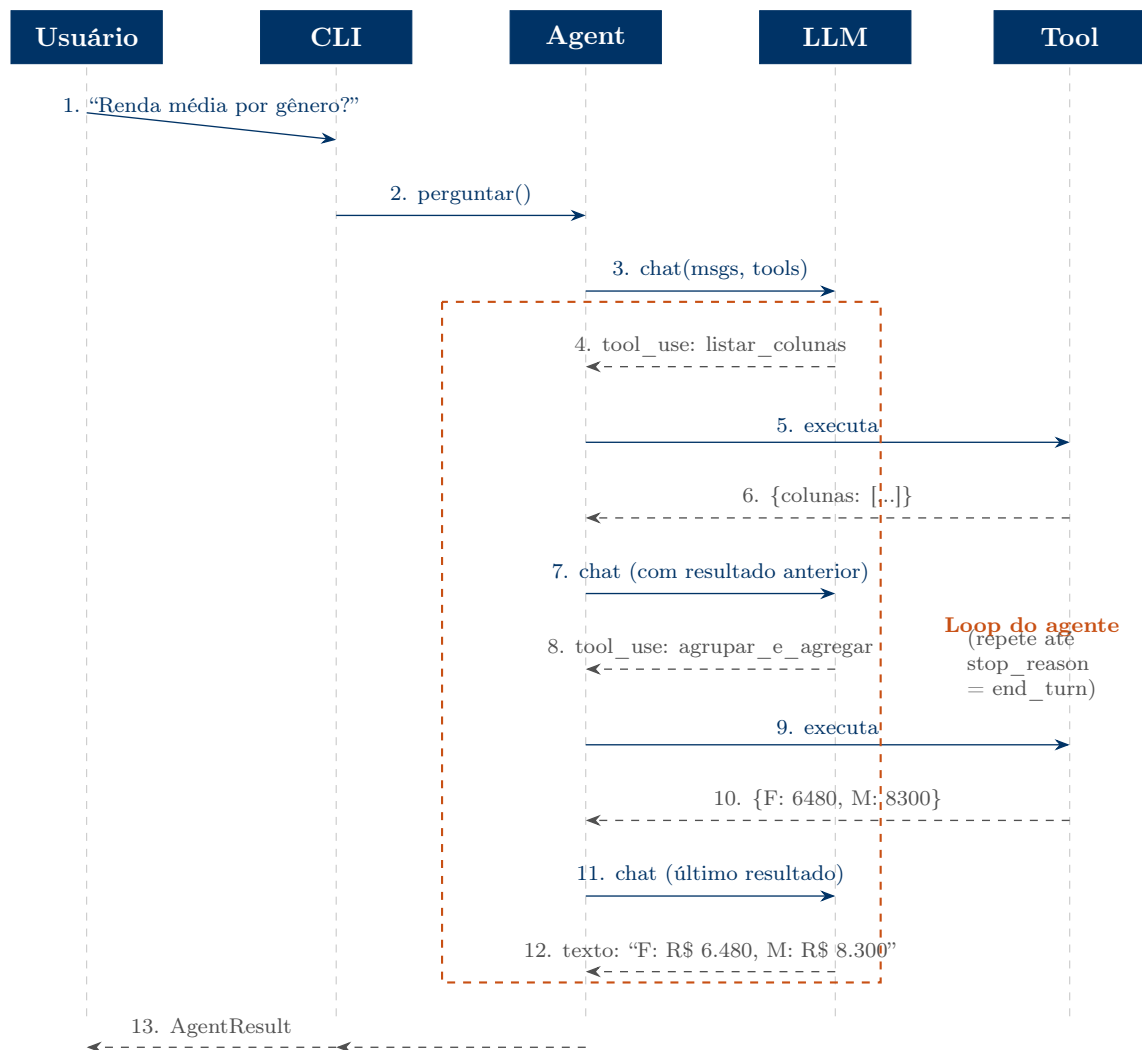


Figura 2: Sequência típica de uma pergunta multi-tool. O loop entre Agent e LLM se repete enquanto o LLM solicitar tools. Setas sólidas: requisições; tracejadas: respostas.

Observação importante: o LLM não tem “memória” entre as chamadas 3, 7 e 11. A cada nova chamada, mandamos o histórico inteiro. Por isso o consumo de tokens cresce a cada iteração do loop — e por isso temos `MAX_AGENT_ITERATIONS` como salvaguarda.

4 Catálogo de artefatos

Esta seção descreve cada artefato do projeto, sua responsabilidade e seus pontos de extensão.

4.1 Estrutura completa de pastas

```
projeto_agente_eda/
|-- README.md          <- Documentacao principal
|-- requirements.txt    <- Dependencias Python
|-- .env.example       <- Template de variaveis de ambiente
|-- .gitignore
|-- config.py          <- Configuracao centralizada
|-- cli.py             <- Entry point (interface interativa)
|
|-- agent/             <- ORQUESTRACAO
|   |-- __init__.py
|   |-- agent.py       <- Loop principal do agente
|   |-- llm_client.py  <- Cliente da API LLM
|
|-- tools/             <- FERRAMENTAS
|   |-- __init__.py    <- Imports que disparam registro
|   |-- base.py        <- @tool decorator e DataState
|   |-- inspect_tools.py <- 3 tools de inspecao
|   |-- filter_tools.py <- 2 tools de filtragem
|   |-- stats_tools.py  <- 2 tools estatisticas
|   |-- plot_tools.py   <- 1 tool de visualizacao
|
|-- evaluation/        <- AVALIACAO
|   |-- __init__.py
|   |-- benchmark.json  <- Conjunto de perguntas + gabarito
|   |-- benchmark.py    <- Executor do benchmark
|   |-- metrics.py      <- Comparadores de resposta
|
|-- tests/             <- TESTES UNITARIOS
|   |-- __init__.py
|   |-- test_tools.py
|
|-- data/              <- DATASETS (CSVs do grupo)
|-- outputs/           <- Graficos gerados pelo agente
'-- logs/              <- Logs do benchmark
```

4.2 Visão por responsabilidade

Arquivo	Responsabilidade	Modificam?
config.py	Configurações centralizadas (caminhos, modelo, limites)	Sim
cli.py	Interface de linha de comando, ponto de entrada	Sim
agent/agent.py	Loop principal: pergunta → LLM → tool → ...	Pouco
agent/llm_client.py	Comunicação com a API	Pouco
tools/base.py	Infraestrutura: @tool, DataState	Não
tools/*_tools.py	Implementação das tools obrigatórias	Sim
evaluation/benchmark.json	Conjunto de avaliação	Sim
evaluation/benchmark.py	Executor do benchmark	Pouco
evaluation/metrics.py	Comparadores de resposta	Pouco
tests/test_tools.py	Testes unitários	Sim

Legenda: Sim = vocês vão alterar com frequência. Pouco = só mexer com bom motivo (refinamento de prompt, adaptação de provedor).

5 Camada 1 – Interface (cli.py)

5.1 O que faz

A CLI é o ponto de entrada do usuário. Carrega o dataset, instancia o agente e fica em um *loop* de leitura: lê uma pergunta, repassa ao agente, imprime o resultado, repete.

5.2 Comandos especiais

Além das perguntas em linguagem natural, a CLI aceita comandos especiais que começam com `/`:

Comando	Uso
<code>/sair</code>	Encerra a sessão
<code>/ajuda</code>	Lista de comandos
<code>/trajetoria</code>	Mostra a trajetória completa da última pergunta
<code>/custo</code>	Acumula tokens e tempo gastos na sessão

5.3 Fluxo de execução

1. Verifica que o CSV existe em `DATASET_PATH`.
2. Chama `state.load()` para carregar o DataFrame em memória.
3. Instancia `Agent()` — isso vai validar a API key.
4. Entra em loop: lê linha do terminal, decide se é comando especial ou pergunta, e despacha.
5. Para perguntas, chama `agente.perguntar(pergunta)` e imprime o resultado dentro de um `Panel` colorido (verde se sucesso, vermelho se falha).

5.4 Pontos de extensão para os alunos

- Acrescentar comando `/recarregar` para trocar de CSV sem reiniciar a CLI.
- Histórico de perguntas salvo em arquivo para resumir uma sessão.
- Substituir CLI por `Streamlit` para uma demo visual mais atraente (a infraestrutura do agente continua a mesma — só muda a interface).

6 Camada 2 – Orquestração

Esta é a camada mais importante — e a mais sutil. Aqui mora o “loop de raciocínio” do agente.

6.1 agent/llm_client.py

6.1.1 Responsabilidade

Encapsular a comunicação com a API do LLM. Toda a especificidade do provedor (formato de mensagem, formato de tool calls, headers) fica isolada aqui.

6.1.2 Por que existe um cliente separado

Se vocês decidirem trocar o provedor (de Anthropic para OpenAI, por exemplo), **só este arquivo precisa mudar**. O `Agent` fala com o cliente através da classe `LLMResponse` (definida no próprio arquivo), que é independente de provedor.

6.1.3 Estrutura

```
@dataclass
class LLMResponse:
    text: str                # Texto livre gerado
    tool_calls: list[dict]   # Tools que o LLM quer chamar
    raw_response: object    # Resposta crua (para debug)
    input_tokens: int        # Tokens enviados
    output_tokens: int       # Tokens gerados
    stop_reason: str        # 'end_turn', 'tool_use', etc.
    latency_seconds: float

class LLMClient:
    def __init__(self, model, api_key): ...
    def chat(self, messages, tools, system="") -> LLMResponse: ...
```

6.1.4 Tratamento de erros

Erros comuns que vocês podem encontrar:

Erro	Causa provável
authentication_error	API key inválida ou ausente no <code>.env</code>
rate_limit_error	Muitas chamadas em pouco tempo (esperar e tentar)
invalid_request_error	Mensagem mal-formada ou tool com schema incorreto
overloaded_error	API sobrecarregada (raro, tentar novamente)

6.2 agent/agent.py

6.2.1 Responsabilidade

Implementar o **loop ReAct** (Reasoning + Acting): conversa com o LLM, executa as tools que ele pede, devolve os resultados, repete até ele dar uma resposta final.

6.2.2 Estruturas principais

```
@dataclass
class Step:
    """Um passo da trajetória do agente."""
    tipo: str                # 'llm_text' / 'tool_call' / 'tool_result'
    conteudo: dict | str
    timestamp: str

@dataclass
class AgentResult:
    """Resultado completo de uma pergunta."""
    pergunta: str
    resposta_final: str
    sucesso: bool
    trajetoria: list[Step]
    total_iteracoes: int
    total_tool_calls: int
    input_tokens: int
    output_tokens: int
    latencia_total: float
```

6.2.3 Anatomia do loop principal

Pseudo-código do método `perguntar()`:

```
def perguntar(self, pergunta):
    messages = [{"role": "user", "content": pergunta}]
    trajetoria = []

    for iteracao in range(MAX_AGENT_ITERATIONS):
        resposta = self.llm.chat(messages, tools, system=PROMPT)

        # Caso 1: LLM terminou de raciocinar -> resposta final
        if resposta.stop_reason == "end_turn":
            return AgentResult(resposta=resposta.text, ...)

        # Caso 2: LLM quer chamar tools
        messages.append({"role": "assistant", "content": resposta.raw})

        resultados = []
        for tc in resposta.tool_calls:
            spec = get_tool_by_name(tc["name"])
            resultado = spec.function(**tc["input"]) # executa!
            resultados.append({
                "type": "tool_result",
                "tool_use_id": tc["id"],
                "content": json.dumps(resultado)
            })

        messages.append({"role": "user", "content": resultados})

    # Saiu do loop sem terminar -> falha
    return AgentResult(sucesso=False, ...)
```

6.2.4 Por que o LLM nunca chama uma tool diretamente

O LLM **nunca** executa código por conta própria. Ele apenas devolve uma estrutura como:

```
{
  "type": "tool_use",
  "id": "toolu_01ABC...",
  "name": "agrupar_e_agregar",
  "input": {
    "grupo": "genero",
    "coluna": "salario",
    "funcao": "mean"
  }
}
```

É o Agent que pega esse **name** + **input**, busca a função correspondente no **TOOL_REGISTRY** e a invoca. Isso é uma decisão de **segurança**: garante que apenas funções pré-aprovadas possam ser executadas.

Atenção

Se vocês forem demonstrar a arquitetura na banca, este é o ponto que mostra maturidade de design. “O LLM não executa código; ele só sugere o que chamar” — essa frase vale 10 minutos de explicação técnica.

6.2.5 O SYSTEM_PROMPT

O **SYSTEM_PROMPT** é o conjunto de *instruções gerais* dadas ao LLM, enviado a CADA chamada. Define a “personalidade” e as regras do agente. Está em `agent/agent.py`, próximo do topo.

Regras importantes ao editar:

- Cada palavra a mais é token a mais por chamada — sejam concisos.

- Diretrizes positivas (“faça X”) funcionam melhor que negativas (“não faça Y”).
- Vocês podem testar mudanças isoladas e medir o impacto na acurácia do benchmark.

7 Camada 3 – Ferramentas (tools/)

7.1 O contrato de uma tool

Toda tool segue o mesmo padrão. Aqui está o esqueleto:

```
@tool(  
    description="Texto curto que o LLM le para decidir se chama esta tool.",  
    parameters={  
        "type": "object",  
        "properties": {  
            "nome_param": {  
                "type": "string",  
                "description": "Para que serve este parametro.",  
            },  
        },  
        "required": ["nome_param"],  
    },  
)  
def nome_da_tool(nome_param: str) -> dict:  
    """Docstring para humanos lerem."""  
    df = state.require_loaded()  
  
    # Validacao defensiva  
    if nome_param not in df.columns:  
        return {"erro": f"Coluna '{nome_param}' nao existe."}  
  
    # Logica  
    resultado = ...  
  
    # Retorno: SEMPRE um dict serializavel para JSON  
    return {"chave": valor, ...}
```

7.2 Por que o retorno é sempre um dict

O resultado da tool é serializado para JSON e enviado de volta ao LLM. Por isso:

- Nada de retornar `DataFrame`, `Series`, ou objetos `numpy`: o JSON não sabe lidar com eles. Conversa para `float`, `int`, `str`, ou estruturas aninhadas de `dict/list`.
- Erros são retornados como `{"erro": "mensagem"}`, não como exceções. Assim o LLM lê a mensagem e pode tentar de outro jeito.
- Números devem ser arredondados (3 a 4 casas decimais) antes do retorno. Um `float` “cru” como `6480.123456789` confunde o LLM e gasta tokens desnecessários.

7.3 Catálogo das 8 tools obrigatórias

Tool	Módulo	O que retorna
listar_colunas	inspect_tools	Nomes e tipos de cada coluna
descrever_dados	inspect_tools	describe() estendido (numéricas + categóricas)
contar_valores	inspect_tools	Distribuição de valores de uma coluna
filtrar	filter_tools	Subconjunto + estatísticas
agrupar_e_agregar	filter_tools	Resultado de groupby + agg
correlacao	stats_tools	Correlação Pearson ou Spearman
detectar_outliers	stats_tools	Outliers por IQR ou z-score
gerar_grafico	plot_tools	Caminho do PNG gerado

7.4 Como adicionar uma tool extra (passo a passo)

Para pontuar no item de bônus da rubrica, sigam exatamente este roteiro:

1. Escolham qual tool faz sentido para o seu dataset. Reflita: “que análise eu queria fazer e o agente não consegue?”
2. Criem um novo arquivo tools/extra_tools.py (ou adicionem em um arquivo existente). Importe tool e state de .base.
3. Implementem a função com o decorador @tool:

```
from .base import tool, state

@tool(
    description="Calcula um percentil especifico de uma coluna numerica.",
    parameters={
        "type": "object",
        "properties": {
            "coluna": {"type": "string"},
            "percentil": {
                "type": "number",
                "description": "Valor entre 0 e 100 (ex: 95)",
            },
        },
        "required": ["coluna", "percentil"],
    },
)
def calcular_percentil(coluna: str, percentil: float) -> dict:
    df = state.require_loaded()
    if coluna not in df.columns:
        return {"erro": f"Coluna '{coluna}' nao existe."}
    if not (0 <= percentil <= 100):
        return {"erro": "Percentil deve estar entre 0 e 100."}
    valor = df[coluna].quantile(percentil / 100)
    return {
        "coluna": coluna,
        "percentil": percentil,
        "valor": round(float(valor), 3),
    }
```

4. Adicionem o import no tools/__init__.py:

```
from . import extra_tools
```

5. Escrevam pelo menos 2 testes em tests/test_tools.py.

6. **Justifiquem no relatório** por que essa tool é útil para o seu dataset e que tipo de pergunta ela viabiliza.

7.5 DataState: por que um estado global

Pode parecer estranho usar um objeto global (**state**) em vez de passar o DataFrame para cada tool. A razão é prática:

- As tools são chamadas pelo **Agent** com base no JSON Schema. Passar o DataFrame como argumento exigiria que ele aparecesse no schema, o que é impossível.
- Carregar o CSV uma vez (em `cli.py`) e manter na memória é milhares de vezes mais rápido do que recarregar a cada chamada.

Nota técnica

Em um projeto de produção, isso seria refatorado para *dependency injection* ou um *context object* passado às tools. Para o escopo deste trabalho, o estado global é a solução pragmática.

8 Camada 4 – Avaliação (evaluation/)

8.1 O conceito de benchmark

O benchmark é o **cerne do rigor científico** do trabalho. Sem ele, vocês têm um sistema que “parece funcionar”; com ele, vocês têm **evidência quantificável** de que funciona.

8.2 Estrutura do `benchmark.json`

Cada pergunta tem este formato:

```
{
  "id": "f-007",
  "tipo": "factual",
  "pergunta": "Qual o salario maximo no dataset?",
  "resposta_esperada": 13500,
  "tipo_resposta": "numero_inteiro",
  "observacao": "Calculado via df['salario'].max()"
}
```

Tipos de resposta aceitos:

tipo_resposta	Como é avaliado
numero_inteiro	Extraí primeiro número da resposta, compara com tolerância
numero_float	Idem
lista_strings	Verifica se todos os itens aparecem na resposta (case-insensitive)
dict_numerico	Verifica chaves e valores numéricos próximos da chave
categorica	Verifica se palavra-chave aparece (inclui recusa)

8.3 Distribuição obrigatória das 30 perguntas



8.4 Pipeline de execução do benchmark

1. `benchmark.py` carrega `benchmark.json`.
2. Para cada pergunta, chama `agente.perguntar(pergunta)`.
3. Compara `resposta_final` com `resposta_esperada` usando o comparador apropriado (de `metrics.py`).
4. Acumula métricas (acertos, tokens, latência, tool calls).
5. Salva log completo em `logs/benchmark_<timestamp>.json` incluindo a trajetória de cada pergunta.
6. Imprime resumo agregado no terminal.

8.5 Métricas calculadas

Quantitativas (automáticas):

- Acurácia geral
- Acurácia por tipo de pergunta
- Taxa de execução bem-sucedida (perguntas que não causaram crash)
- Número médio de *tool calls* por pergunta
- Latência média por pergunta
- Total de tokens (input e output)

Qualitativas (você escrevem no relatório):

- Análise de 3+ trajetórias completas
- Identificação de 3+ padrões de erro
- Discussão sobre alucinação
- Discussão de custo, latência e implicações éticas

8.6 Limitações dos comparadores automáticos

Os comparadores em `metrics.py` são **pragmáticos, não perfeitos**. Casos onde eles podem falhar:

- Resposta correta em linguagem diferente: “O total é 7” vs. gabarito 7 — funciona porque extraímos o primeiro número.
- Resposta correta com unidades: “R\$ 6.480,00” vs. 6480 — funciona porque normalizamos separadores.
- Resposta *parcialmente* correta: “A média é alta” quando esperávamos um número — conta como erro (não há número extraível).

Discussão no relatório: comentem essas limitações. Considerem propor *LLM-as-judge* (usar outro LLM para julgar a equivalência semântica) como trabalho futuro.

9 Fluxos de uso comuns

9.1 Fluxo 1: Sessão interativa

```
# 1. Ativar o virtualenv (PyCharm faz automaticamente)
source venv/bin/activate

# 2. Conferir que .env esta configurado
cat .env # deve conter ANTHROPIC_API_KEY=sk-ant-...

# 3. Conferir que dataset existe
ls data/ # deve listar seu .csv

# 4. Rodar a CLI
python cli.py

# 5. Fazer perguntas...
> Quais sao as colunas?
> Qual a media de salario por genero?
> /trajetoria          # ver o que o agente fez
> /custo               # ver gasto acumulado
> /sair
```

9.2 Fluxo 2: Rodar testes unitários

```
# Rodar todos
pytest tests/ -v

# Rodar apenas testes de uma tool especifica
pytest tests/test_tools.py::test_correlacao_idade_salario -v

# Rodar com cobertura (precisa de pytest-cov)
pytest --cov=tools --cov=agent tests/
```

9.3 Fluxo 3: Executar o benchmark completo

```
# Garantir que benchmark.json esta preenchido
cat evaluation/benchmark.json | python -m json.tool | head -20

# Executar benchmark
python -m evaluation.benchmark

# Ver o log gerado
ls logs/
cat logs/benchmark_<timestamp>.json | python -m json.tool | less
```

9.4 Fluxo 4: Workflow de desenvolvimento (ciclo curto)

Ao implementar uma nova tool ou corrigir um bug:

1. Edite o código da tool em `tools/`.
2. Adicione um teste em `tests/test_tools.py`.
3. Rode `pytest tests/test_tools.py::seu_teste -v`.
4. Quando o teste passar, rode `python cli.py` e teste interativamente.
5. Quando estiver feliz, commit no Git.

Nota técnica

Não chamem o LLM durante desenvolvimento toda hora. **Cada chamada custa tokens.** Use pytest com mocks ou prints diretos das funções enquanto estiver implementando. Só rode o agente full quando a lógica básica estiver sólida.

10 Modelo de custo e otimização

10.1 De onde vem o custo

O preço da API é cobrado por token, com tarifas diferentes para input (que vocês enviam) e output (que o modelo gera). Para Claude Haiku, no momento desta documentação:

- Input: aproximadamente US\$ 0,80 por milhão de tokens
- Output: aproximadamente US\$ 4,00 por milhão de tokens

Verifiquem o preço atual em <https://www.anthropic.com/pricing> antes de fechar o relatório.

10.2 Onde os tokens vão

Em cada iteração do loop do agente, o LLM recebe:

1. O SYSTEM_PROMPT (constante, ~200 tokens)
2. A descrição de todas as tools (constante, ~500–800 tokens)
3. A pergunta original do usuário (~20 tokens)
4. TODOS os resultados de tools anteriores neste loop (varia muito)

Em uma pergunta que requer 3 tool calls, o agente faz 4 chamadas ao LLM, e em cada uma o *histórico inteiro* é re-enviado. Daí o custo cresce mais que linearmente.

10.3 Estratégias de otimização

Estratégia	Como aplicar
Resumir respostas de tools	Limitar tamanho de listas, arredondar números, omitir campos pouco usados
Prompt enxuto	Cada linha do SYSTEM_PROMPT deve justificar seu peso em tokens
Tools com escopo claro	Tools muito genéricas levam o LLM a fazer “viagens” — prefiram tools específicas
Limitar MAX_ITERATIONS	10 é generoso. Para muitos datasets, 5 já basta
Usar modelo menor	Para a maioria das perguntas, Haiku resolve. Sonnet é overkill

11 Glossário

Termo	Definição
Agente	Sistema que usa um LLM para decidir, a cada passo, o que fazer (em vez de seguir um fluxo fixo).
Tool / Ferramenta	Função Python registrada que o LLM pode pedir para executar.
Tool use / Function calling	Capacidade do LLM de retornar uma estrutura indicando “chame essa função com esses argumentos”.
System prompt	Instruções gerais dadas ao LLM no início de toda interação, definindo seu comportamento.
Trajetória	Sequência completa de passos (texto + tool calls + resultados) que o agente executou para responder a uma pergunta.
Stop reason	Razão pela qual o LLM parou: end_turn (resposta final), tool_use (quer chamar tool), max_tokens (estourou limite).
Benchmark	Conjunto fixo de perguntas com respostas conhecidas, usado para medir o desempenho do agente de forma reproduzível.
Alucinação	Quando o LLM “inventa” um fato (uma coluna inexistente, um valor não calculado).
LLM-as-judge	Técnica em que outro LLM avalia a equivalência semântica entre uma resposta e um gabarito.